20

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

동적 모듈과 글로벌 모듈

제2부 NestJS 핵심 · 5장 의존성 주입

같은 모듈을 환경마다 다른 설정으로, 설정을 읽는 시점



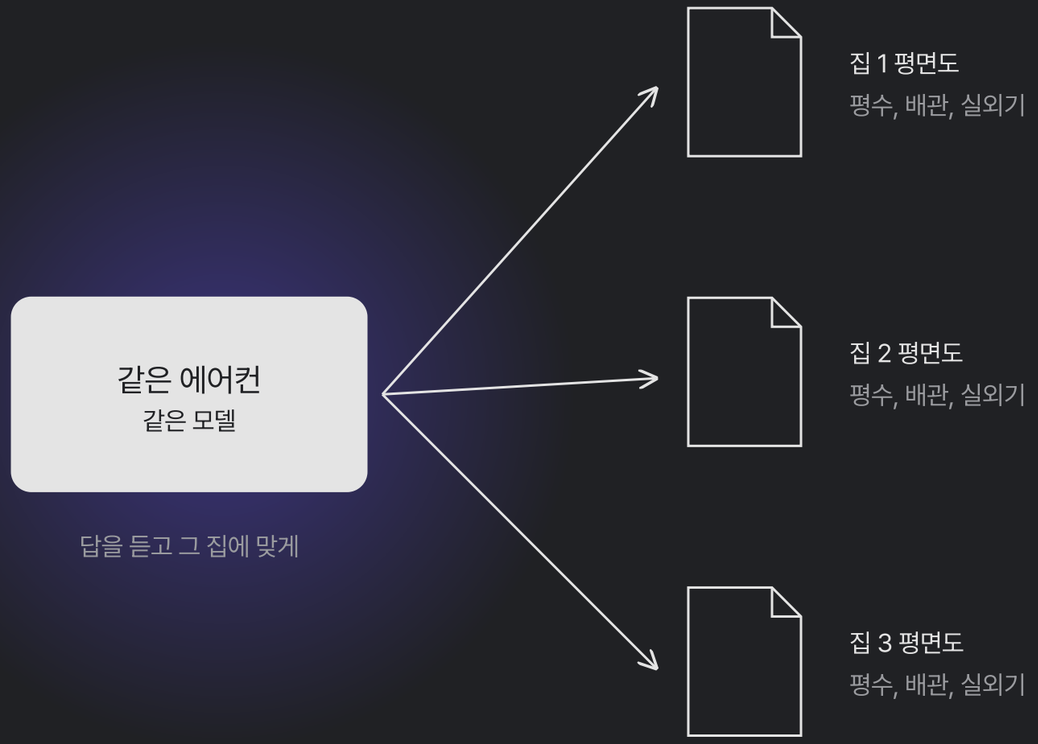
© 한상훈 NestJS 백엔드 마스터

에어컨 설치 기사

→ 같은 모델, 집집마다 다른 설치

→ 기사가 묻는 것: 평수, 배관 길이, 실외기 위치

→ 답을 듣고 그 집에 맞게 설정



© 한상훈 NestJS 백엔드 마스터

코드에 박힌 결제사 주소

payments.module.ts

```
@Module({
  providers: [
    {
      provide: PAYMENT_OPTIONS,
      useValue: { baseUrl: 'http://localhost:5018', timeoutMs: 3000 },
    },
    ...
  ],
})
export class PaymentsModule {}
```

→ 주소를 바꾸려면 결제 모듈 코드 수정

© 한상훈 NestJS 백엔드 마스터

환경마다 다른 값

환경	결제사 주소	대기 시간
로컬	가짜 결제사 서버	3초
스테이징	결제사 샌드박스	3초
운영	결제사 운영 API	운영 기준

→ 같은 결제 모듈, 다른 설정

© 한상훈 NestJS 백엔드 마스터

모듈을 돌려주는 함수

payments.module.ts

```
@Module({})
export class PaymentsModule {
  static forRoot(options: PaymentOptions): DynamicModule {
    return {
      module: PaymentsModule,
      global: true,
      providers: [
        { provide: PAYMENT_OPTIONS, useValue: options },
        gatewayProvider,
        { provide: 'LEGACY_PAYMENT', useExisting: PaymentGateway },
      ],
      exports: [PaymentGateway, 'LEGACY_PAYMENT'],
    };
  }
}
```

→ `gatewayProvider`: 앞에서 만든 useFactory 프로바이더 그대로 (부록)

→ `@Module({})`: 빈 메타데이터, 내용은 forRoot가 반환

© 한상훈 NestJS 백엔드 마스터

루트 모듈에서 설정 넘기기

app.module.ts

```
@Module({
  imports: [
    PaymentsModule.forRoot({
      mode: process.env.PAYMENT_MODE === 'fake' ? 'fake' : 'http',
      baseUrl: process.env.PG_URL,
      timeoutMs: Number(process.env.PG_TIMEOUT_MS ?? 3000),
    }),
    ConcertsModule,
    PerformancesModule,
    SeatsModule,
    ReservationsModule,
  ],
})
export class AppModule {}
```

→ PaymentOptions에 mode 추가: `'fake' | 'http'`

© 한상훈 NestJS 백엔드 마스터

동적 모듈

→ 동적 모듈(dynamic module): 실행 중에 함수가 만들어 돌려주는 모듈 설정

→ `DynamicModule`: module + providers, exports, imports, global

→ 정적 모듈: `@Module`에 내용이 고정

→ 설정을 받아 그 설정이 든 프로바이더를 만드는 모듈

© 한상훈 NestJS 백엔드 마스터

.env와 loadEnvFile

```
# .env
PG_URL=http://localhost:5020
PG_TIMEOUT_MS=3000
```

→ `process.loadEnvFile()`: Node 내장, .env를 `process.env`로

main.ts

```ts
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module.js';

process.loadEnvFile();

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
await bootstrap();
```

© 한상훈 NestJS 백엔드 마스터

.env에 있는데 undefined

부팅 로그

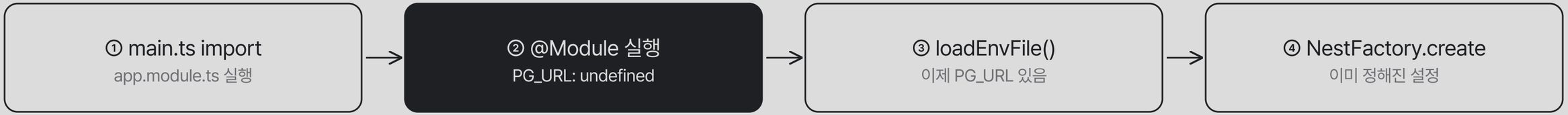
```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
결제 설정: {"mode":"http","timeoutMs":3000}
TaggedLogger #1 생성
TaggedLogger #2 생성
SeatsService 생성
ERROR [ExceptionHandler] TypeError: Failed to parse URL from undefined/token
  [cause]: TypeError: Invalid URL
    code: 'ERR_INVALID_URL',
    input: 'undefined/token'
```

→ 결제 설정에 baseUrl 없음

→ timeoutMs 3000: .env 값이 아닌 코드의 기본값

© 한상훈 NestJS 백엔드 마스터

먼저 실행되는 import



→ ① main.ts의 import 문 → app.module.ts 실행

→ ② @Module 데코레이터 실행 → `forRoot({ baseUrl: process.env.PG_URL })` 계산: undefined

→ ③ main.ts 본문: `process.loadEnvFile()` → 이제 PG_URL 있음

→ ④ NestFactory.create → 이미 undefined로 정해진 설정

→ ES 모듈: import한 파일을 본문보다 먼저 끝까지 실행

© 한상훈 NestJS 백엔드 마스터

forRootAsync

payments.module.ts

```
static forRootAsync(options: {
  imports?: ModuleMetadata['imports'];
  inject?: InjectionToken[];
  useFactory: (...args: any[]) => PaymentOptions | Promise<PaymentOptions>;
}): DynamicModule {
  return {
    module: PaymentsModule,
    global: true,
    imports: options.imports ?? [],
    providers: [
      {
        provide: PAYMENT_OPTIONS,
        inject: options.inject ?? [],
        useFactory: options.useFactory,
      },
      gatewayProvider,
      { provide: 'LEGACY_PAYMENT', useExisting: PaymentGateway },
    ],
    exports: [PaymentGateway, 'LEGACY_PAYMENT'],
  };
}
```

→ 설정 값 대신 설정을 만드는 함수를 받음

© 한상훈 NestJS 백엔드 마스터

루트 모듈의 forRootAsync

app.module.ts

```
PaymentsModule.forRootAsync({
  useFactory: () => ({
    mode: process.env.PAYMENT_MODE === 'fake' ? 'fake' : 'http',
    baseUrl: process.env.PG_URL,
    timeoutMs: Number(process.env.PG_TIMEOUT_MS ?? 3000),
  }),
}),
```

→ 화살표 함수 안의 `process.env`: Nest가 이 함수를 부를 때 읽음

© 한상훈 NestJS 백엔드 마스터

이제 찍힌 결제사 주소

부팅 로그와 예매 요청 두 번

```
LOG [NestFactory] Starting Nest application...
ConcertsService 생성
결제 설정: {"mode":"http","baseUrl":"http://localhost:5020","timeoutMs":3000}
TaggedLogger #1 생성
TaggedLogger #2 생성
SeatsService 생성
HttpPaymentGateway 생성: 토큰 tok-123
LOG [NestApplication] Nest application successfully started

{"seatId":"A-3-15","approvalId":"pg-110000","amount":110000,"notifyTo":"김민수"}
{"seatId":"B-7-2","approvalId":"pg-99000","amount":99000,"notifyTo":"이지은"}
```

→ 5020번: 이 강의 가짜 결제사 서버

© 한상훈 NestJS 백엔드 마스터

imports와 inject

→ `inject`: 팩토리 인자로 받을 토큰 목록 (useFactory 프로바이더와 같은 규칙)

→ `imports`: 그 토큰을 가진 모듈을 결제 모듈 안으로 가져옴

→ 예: 설정을 읽어 주는 서비스가 따로 있으면 그 서비스를 inject로 받아 값 조립

→ 이 프로젝트: 아직 설정 서비스 없음, `process.env`를 팩토리 안에서 읽음

© 한상훈 NestJS 백엔드 마스터

register, forRoot, forFeature

이름	관례	예
`forRoot / forRootAsync`	서버에 한 번, 루트에서 설정	DB 연결, 결제사 설정
`register / registerAsync`	가져오는 곳마다 다른 설정	모듈마다 다른 HTTP 클라이언트
`forFeature`	forRoot로 만든 것 중 이 모듈이 쓸 부분	모듈별 엔티티 저장소

→ 이름은 관례, 동작은 반환하는 DynamicModule이 결정

데이터 블록: 공식 패키지 예(TypeOrmModule.forRoot / forFeature, JwtModule.register)와 ConfigurableModuleBuilder는 부록, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

global: true

reservations.module.ts 변경

```
 // reservations.module.ts
-import { PaymentsModule } from '../payments/payments.module.js';

 @Module({
-  imports: [PaymentsModule],
   controllers: [ReservationsController],
   providers: [ReservationsService, RefundsService, TaggedLogger],
 })
 export class ReservationsModule {}
```

→ 결제 모듈 forRoot 반환값의 `global: true`

→ 예매 모듈 imports에 결제 모듈 없음 → 그래도 PaymentGateway 주입

© 한상훈 NestJS 백엔드 마스터

글로벌을 끄면

```
ERROR [ExceptionHandler] UnknownDependenciesException [Error]: Nest can't resolve dependencies of the
ReservationsService (?, TaggedLogger). Please make sure that the argument PaymentGateway at index [0]
is available in the ReservationsModule module.
```

→ `global: false` → 예매 모듈에서 PaymentGateway를 못 찾음

© 한상훈 NestJS 백엔드 마스터

글로벌 모듈

→ 글로벌 모듈(global module): 한 번 등록하면 imports 없이 모든 모듈에서 exports를 쓸 수 있는 모듈

→ 방법: `@Global()` 데코레이터, 또는 DynamicModule의 `global: true`

→ 등록은 한 번: 보통 루트 모듈 imports

→ 맞는 곳: 설정, DB 연결, 결제사처럼 거의 모든 모듈이 쓰는 것

→ 대가: 예매 모듈 코드만 봐서는 결제 모듈에 기대는지 안 보임

© 한상훈 NestJS 백엔드 마스터

예매 모듈의 forRoot 한 줄

reservations.module.ts

```
@Module({
  imports: [PaymentsModule.forRoot({ mode: 'fake', timeoutMs: 1000 })],
  controllers: [ReservationsController],
  providers: [ReservationsService, RefundsService, TaggedLogger],
})
export class ReservationsModule {}
```

→ 예매 모듈만 가짜로 시험하려고 넣은 한 줄이라고 가정

© 한상훈 NestJS 백엔드 마스터

결제 설정 두 줄

부팅 로그와 예매 요청 두 번

```
결제 설정: {"mode":"http","baseUrl":"http://localhost:5020","timeoutMs":3000}
결제 설정: {"mode":"fake","timeoutMs":1000}
FakePaymentGateway #1 생성
HttpPaymentGateway 생성: 토큰 tok-123
LOG [NestApplication] Nest application successfully started

{"seatId":"A-3-15","approvalId":"fake-110000","amount":110000,"notifyTo":"김민수"}
{"seatId":"B-7-2","approvalId":"fake-99000","amount":99000,"notifyTo":"이지은"}
```

→ 운영 설정(http)으로 띄웠는데 예매는 가짜 승인

→ 에러 없음, 부팅 정상

© 한상훈 NestJS 백엔드 마스터

설정이 다르면 다른 모듈

→ forRoot 호출 둘, 설정 둘 → 결제 모듈 인스턴스 둘

→ 예매 모듈: 자기 imports의 결제 모듈(fake)을 먼저 사용

→ 나머지 모듈: 루트의 글로벌 결제 모듈(http)

→ 루트 설정 모듈은 루트에서 한 번, 테스트용 교체는 테스트 쪽에서

데이터 블록: docs.nestjs.com/modules "Calling forRoot() again in another module would create a second, separately configured instance." (2026-10-26 기준), 촬영 직전 확인

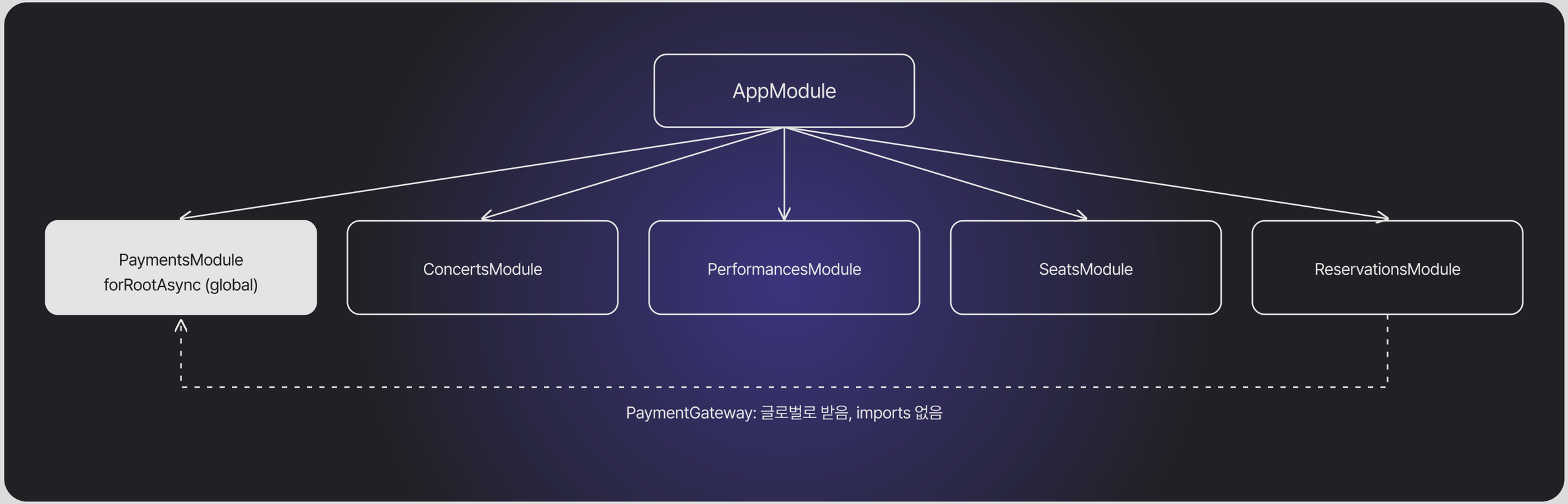
© 한상훈 NestJS 백엔드 마스터

지금의 모듈 그래프

→ AppModule → PaymentsModule.forRootAsync (global)

→ AppModule → ConcertsModule, PerformancesModule, SeatsModule, ReservationsModule

→ ReservationsModule ⇢ PaymentGateway (글로벌로 받음, imports 없음)



© 한상훈 NestJS 백엔드 마스터

설정을 읽는 때와 곳

→ 때: 모듈 파일을 import할 때가 아닌, Nest가 앱을 만들 때

→ 곳: 루트 모듈 한 군데

"설정은 Nest 앱이 만들어질 때 읽고, 루트에서 한 번만 등록합니다."